

Zenora System Design And TypeScript Principles Guide

Generated for the Zenora full-stack project.

This document has two goals:

1. Explain the engineering and system design principles used in modern TypeScript applications.
2. Show exactly how those ideas appear in this project.

The project is a mental healthcare platform with a Next.js frontend and an Express, Prisma, PostgreSQL backend. It includes authentication, therapist discovery, availability, booking, real-time chat, payments, subscriptions, earnings, assessments, treatment plans, prescriptions, complaints, notifications, admin analytics, and crisis monitoring.

1. Project Architecture Overview

Zenora is split into two main applications:

- frontend/: Next.js, React, TypeScript, Tailwind CSS, shadcn-style UI.
- backend/: Express, TypeScript, Prisma, PostgreSQL, Redis, Socket.io, Stripe, Cloudinary.

The backend follows a layered architecture:

```
HTTP Request
-> Route
-> Middleware
-> Controller
-> Service
-> Repository
-> Database / External Provider
-> Response
```

Example:

```
POST /api/v1/sessions
-> SessionRoutes
-> AuthMiddleware + ValidationMiddleware
-> SessionController.book()
-> SessionService.book()
-> SessionRepository / AvailabilitySlotRepository
-> Prisma/PostgreSQL
```

The frontend follows a component and API-client architecture:

```
User Interaction
-> React component
-> Auth context / API helper
-> Backend endpoint
-> UI result / toast / rendered data
```

2. Core System Design Principles

Separation Of Concerns

Separation of concerns means each part of the system has one clear responsibility.

In Zenora:

- Routes define URLs and middleware.
- Controllers translate HTTP requests into service calls.
- Services contain business rules.
- Repositories handle persistence.
- Entities model domain behavior.
- Validators check input shape.
- Middleware handles cross-cutting concerns such as authentication and validation.

This keeps the system easier to change because business logic is not mixed directly into routes or UI code.

Layered Architecture

Layered architecture divides the application into independent layers.

Common layers:

- Presentation layer: frontend UI or HTTP controllers.
- Application layer: services and use cases.
- Domain layer: entities and business rules.
- Infrastructure layer: database, Redis, Stripe, Cloudinary, SMTP.

Zenora uses this clearly in the backend:

- routes/
- controllers/
- services/
- repositories/
- entities/
- middlewares/
- validators/
- config/

Modularity

Modularity means a feature is split into a self-contained module.

Examples in Zenora:

- Auth module
- Therapist module
- Availability module
- Session module
- Chat module
- Payment module
- Subscription module
- Assessment module
- Prescription module
- Complaint module

- Notification module
- Admin analytics module

Each module generally has routes, controllers, services, validators, and sometimes repositories/entities.

Encapsulation

Encapsulation means internal details are hidden behind public methods.

Example:

User hides password hashing details behind methods such as `verifyPassword()` and `setPassword()`.

Session hides lifecycle rules behind methods such as `confirm()`, `start()`, `complete()`, `cancel()`, and `reschedule()`.

This prevents random code from changing critical state without business-rule checks.

Abstraction

Abstraction means exposing a simpler interface over complex implementation details.

Examples:

- `UploadService` hides Cloudinary upload details.
- `PaymentService` hides Stripe details.
- `EmailService` hides SMTP details.
- `DatabaseService` hides Prisma client initialization.
- `ApiResponse` hides response formatting.

Dependency Direction

Higher-level application logic should not depend on HTTP or database details more than necessary.

In Zenora, controllers depend on services, services depend on repositories/config providers, and repositories depend on Prisma.

This is healthier than letting every controller call Prisma directly.

Single Source Of Truth

A system should avoid duplicating important state in multiple conflicting places.

Examples:

- Prisma schema is the database source of truth.
- Auth context is the frontend session source of truth.
- Entity methods are the source of truth for lifecycle rules such as session status transitions.

Stateless API With Token-Based Auth

The backend uses JWT access tokens for API authentication. This allows requests to be stateless: each request carries its own authentication proof.

Refresh-token revocation uses Redis so logout and refresh rotation can invalidate tokens.

Defense In Depth

Security is implemented in multiple layers:

- Helmet for HTTP security headers.
- CORS configuration.
- Rate limiting.
- JWT authentication middleware.

- Role authorization middleware.
- Zod validation.
- Prisma queries instead of string-built SQL.
- Password hashing using bcrypt.

Input Validation

Input validation ensures the server receives data in the expected shape.

Zenora uses Zod validators before controllers call services.

Examples:

- Auth payloads.
- Payment payloads.
- Therapist profile payloads.
- Assessment responses.
- Complaint actions.
- Notification actions.

Transactional Integrity

Some changes must happen together or not at all.

Examples:

- Booking a session and marking a slot booked.
- Rescheduling a session, freeing the old slot, and locking the new slot.
- Creating a rating and recalculating therapist average rating.

These flows use Prisma transactions.

Observability

Observability is the ability to understand what the system is doing.

Zenora includes:

- Request logging middleware.
- Winston logger.
- Error logs.
- Stripe webhook logging.
- Notification dispatch logging.
- Socket connection logging.

Scalability

Scalability means the system can handle growth.

Zenora has early scalability foundations:

- Stateless JWT authentication.
- Redis support.
- Separate frontend/backend applications.
- Database indexes in Prisma schema.
- Pagination in list endpoints.
- Socket.io room-based chat.
- External providers for files, payments, and email.

Maintainability

Maintainability comes from readable structure, consistent naming, small modules, and tests.

Zenora supports maintainability through:

- Consistent file naming.
 - Repeated route/controller/service patterns.
 - Typed entities.
 - Zod validation.
 - Jest tests for entities and validators.
-

3. SOLID Principles

SOLID is a set of object-oriented design principles that make code easier to maintain and extend.

S: Single Responsibility Principle

A class or module should have one reason to change.

Bad example:

```
class UserManager {
  createUser() {}
  sendEmail() {}
  uploadAvatar() {}
  chargeCard() {}
}
```

Better:

```
class UserService {}
class EmailService {}
class UploadService {}
class PaymentService {}
```

In Zenora:

- AuthService handles auth workflows.
- UploadService handles file uploads.
- PaymentService handles payment workflows.
- NotificationService handles notifications.
- SessionService handles session lifecycle.

O: Open/Closed Principle

Software should be open for extension but closed for modification.

Meaning: you should be able to add behavior without rewriting stable existing logic.

Example in Zenora:

Adding a new route class such as RatingRoutes or ComplaintRoutes extends the app by mounting another module in server.ts, without rewriting App itself.

L: Liskov Substitution Principle

If a class extends another class, it should be usable anywhere the parent class is expected without breaking

behavior.

Zenora does not heavily use inheritance, which is often good in service-layer applications. It relies more on composition.

Example concept:

```
class PaymentProvider {
    charge() {}
}

class StripeProvider extends PaymentProvider {
    charge() {}
}
```

If StripeProvider cannot behave like a PaymentProvider, it violates Liskov substitution.

I: Interface Segregation Principle

Do not force code to depend on methods it does not use.

Bad:

```
interface HugeService {
    login(): void;
    upload(): void;
    refund(): void;
    chat(): void;
}
```

Better:

```
interface AuthServiceContract {
    login(): void;
}

interface UploadServiceContract {
    upload(): void;
}
```

In Zenora:

- Route classes implement a small Routes interface with only path and router.
- Feature payload interfaces are specific, such as BookSessionPayload, RefundInput, and CreateTherapistProfilePayload.

D: Dependency Inversion Principle

High-level modules should not depend directly on low-level details. Both should depend on abstractions.

In Zenora:

Services often receive repositories through constructor defaults:

```
constructor(private readonly users: UserRepository = userRepository) {}
```

This makes it easier to replace the repository in tests or future implementations.

4. TypeScript Fundamentals

What Is TypeScript?

TypeScript is JavaScript with static types. It lets you describe the shape of values before runtime.

Benefits:

- Catch mistakes earlier.
- Improve autocomplete.
- Make APIs easier to understand.
- Refactor with more confidence.

What Is A Type?

A type describes what values are allowed.

```
type Role = "CLIENT" | "THERAPIST" | "ADMIN";
```

This means a variable of type Role can only be one of those three strings.

What Is An Interface?

An interface describes the shape of an object.

```
interface UserResponse {  
  id: string;  
  name: string;  
  email: string;  
}
```

If a value claims to be UserResponse, it must have those fields.

Interfaces are often used for:

- DTOs.
- Function parameters.
- Class contracts.
- Shared object shapes.

Example from Zenora:

```
export interface Routes {  
  path: string;  
  router: Router;  
}
```

Any route class that implements Routes must provide path and router.

Type Alias vs Interface

Both can describe object shapes.

```
type User = {  
  id: string;  
};  
  
interface User {  
  id: string;  
}
```

Common preference:

- Use interface for object contracts that classes may implement.
- Use type for unions, primitives, function signatures, or composed types.

Example:

```
type SessionStatus = "PENDING" | "CONFIRMED" | "COMPLETED";
```

What Is A Class?

A class is a blueprint for creating objects that hold data and behavior.

```
class Session {  
  public status: string;  
  
  public confirm() {  
    this.status = "CONFIRMED";  
  }  
}
```

Classes are useful when state and behavior belong together.

In Zenora, domain entities such as User, Session, Payment, and Assessment are classes.

What Is An Object?

An object is an actual value created from a class or literal.

```
const session = new Session();
```

What Is A Constructor?

A constructor runs when a class instance is created.

```
class User {  
  constructor(public name: string) {}  
}
```

In Zenora, constructors initialize entity state.

Public, Private, And Protected

TypeScript access modifiers control where class members can be accessed.

public

Accessible from anywhere.

```
class User {  
  public name: string;  
}
```

If no modifier is written, TypeScript treats it as public by default.

private

Accessible only inside the same class.

```
class User {  
  private passwordHash: string;  
}
```

Can an inherited class access private members?

No. A subclass cannot access private members directly.

```
class Parent {  
  private secret = "hidden";  
}  
  
class Child extends Parent {  
  read() {  
    // this.secret is not allowed  
  }  
}
```

protected

Accessible inside the same class and subclasses, but not from outside.

```
class Parent {  
  protected token = "abc";  
}  
  
class Child extends Parent {  
  read() {  
    return this.token;  
  }  
}
```

Can an inherited class access protected members?

Yes. Subclasses can access protected members.

readonly

Readonly means a property can be assigned during initialization but cannot be reassigned later.

```
class Session {  
  public readonly id: string;  
}
```

In Zenora, many entity IDs are readonly because identity should not change after creation.

Static Methods

A static method belongs to the class itself, not an instance.

```
class User {  
  static normalizeEmail(email: string) {  
    return email.toLowerCase();  
  }  
}
```

In Zenora:

- User.create()
- User.fromPersistence()
- Assessment.scoreToSeverity()
- DatabaseService.getInstance()

Inheritance vs Composition

Inheritance means one class extends another.

Composition means one class uses another object to do work.

Zenora mostly uses composition:

```
class AuthService {  
  constructor(private readonly users: UserRepository = userRepository) {}  
}
```

This is often better for backend services because it reduces tight coupling.

Generics

Generics allow reusable types.

```
class ApiResponse<T> {  
  data: T | null;  
}
```

T is a placeholder. It can become User, Payment, Session, etc.

DTOs

DTO means Data Transfer Object. It is the shape of data moving between layers or over the network.

Examples:

- Login payload.

- User response.
- Session booking payload.
- Assessment submission payload.

Entity

An entity is a domain object with identity and behavior.

Examples:

- User
- Session
- Therapist
- Payment
- Assessment

Repository

A repository hides database access.

Instead of services writing raw Prisma queries everywhere, they call repository methods.

Examples:

- UserRepository.findByEmail()
- SessionRepository.findById()
- PaymentRepository.list()

Service

A service contains business logic.

Examples:

- AuthService.login()
- SessionService.book()
- PaymentService.refund()
- ComplaintService.resolve()

Middleware

Middleware runs before the final request handler.

Examples:

- Authentication middleware.
- Authorization middleware.
- Validation middleware.
- Rate limiting middleware.
- Error handling middleware.

5. Backend Design Topics

REST API

REST APIs expose resources over HTTP.

Examples:

- POST /auth/login
- GET /therapists
- POST /sessions
- PATCH /sessions/:sessionId/confirm
- POST /payments

Authentication

Authentication answers: who are you?

Zenora supports:

- Email/password login.
- JWT access tokens.
- Refresh tokens.
- Google OAuth.
- Password reset.
- Email verification.

Authorization

Authorization answers: what are you allowed to do?

Zenora uses role-based authorization:

- CLIENT
- THERAPIST
- ADMIN

Examples:

- Only clients can book sessions.
- Only therapists can confirm/start/complete sessions.
- Only admins can approve therapists.

JWT

JWT means JSON Web Token. It is a signed token containing claims such as user ID and role.

Zenora uses:

- Access token for API calls.
- Refresh token for issuing new access tokens.
- Redis to revoke refresh tokens.

OAuth

OAuth lets users authenticate through an external provider such as Google.

Zenora has Google OAuth start and callback endpoints.

Password Hashing

Passwords should never be stored in plain text.

Zenora hashes passwords with bcrypt.

Validation With Zod

Zod validates runtime data.

TypeScript only checks code at compile time. Zod checks incoming request data at runtime.

ORM And Prisma

Prisma maps TypeScript code to database operations.

Benefits:

- Type-safe database queries.
- Schema migrations.
- Relation modeling.
- Easier CRUD.

Database Indexing

Indexes make queries faster.

Zenora uses indexes in Prisma schema for common lookups such as therapist verification status, specialization, rating, session status, and chat messages by session/time.

Transactions

Transactions ensure several database operations succeed or fail together.

Examples:

- Booking session + marking slot booked.
- Rescheduling session + moving slot booking.
- Rating therapist + recalculating average.

Webhooks

Webhooks let external services notify the backend.

Zenora uses Stripe webhooks for payment events.

File Storage

Zenora uses Cloudinary for:

- Avatars.
- Therapist documents.
- Prescription PDFs.
- Misc uploads.

Real-Time Communication

Zenora uses Socket.io for chat.

Concepts:

- A socket connection stays open.
- Users join session-specific rooms.
- Messages are emitted to a room.
- Read receipts and typing indicators are real-time events.

Background / Best-Effort Work

Some operations should not block the main request.

Examples:

- Prescription PDF generation is best-effort.
- Notification dispatch logs provider failures without crashing all flows.

Error Handling

Zenora uses `ApiError` to standardize errors and global error middleware to format responses.

Pagination

List endpoints use `page/limit` parameters.

This avoids returning too much data at once.

Rate Limiting

Rate limiting protects the API from abuse.

Zenora applies global request rate limiting.

6. Frontend Design Topics

Component Architecture

The frontend uses React components.

Examples:

- `AuthScreen`
- `AppDashboard`
- `ThemeToggle`
- UI primitives such as `Button`, `Card`, `Input`, `Select`, and `Badge`

Client State

The frontend uses `AuthContext` to store:

- Current user.
- Access token.
- Refresh token.
- `Login/register/logout` functions.

API Client Abstraction

The frontend uses `apiRequest()` and `apiUpload()` to centralize fetch logic.

Benefits:

- One place for base URL.
- One place for auth headers.
- One place for error handling.

Theming

The frontend supports:

- Light mode.
- Dark mode.
- Night mode.

It uses CSS variables inspired by `shadcn UI`.

Role-Aware UI

The UI changes based on user role:

- Client dashboard.
- Therapist dashboard.
- Admin dashboard.

Form-Driven Operations

Most backend features are exposed as simple operational forms. This is useful for early product development because it proves that the backend workflows are callable before building highly customized screens.

7. Principles Used In This Project And Where

Routes Interface

File:

- backend/src/interfaces/route.interface.ts

Principles:

- Interface Segregation.
- Open/Closed.
- Consistent contracts.

Every route class exposes a path and router, so the server can mount all modules in the same way.

App Composition

File:

- backend/src/server.ts

Principles:

- Composition over inheritance.
- Open/Closed.
- Modular architecture.

New features are added by creating route classes and passing them into new App([...]).

Express App Setup

File:

- backend/src/app.ts

Principles:

- Separation of concerns.
- Middleware pipeline.
- Centralized error handling.

The app configures security middleware, body parsing, logging, rate limiting, routes, 404 handling, and global error handling.

Auth Middleware

File:

- backend/src/middlewares/AuthMiddleware.ts

Principles:

- Defense in depth.
- Single responsibility.
- Role-based authorization.

authenticate verifies JWTs. authorize checks roles.

Validation Middleware

File:

- backend/src/middlewares/validate.ts

Principles:

- Input validation.
- DRY.
- Runtime safety.

All request validation is centralized instead of being duplicated in every controller.

Entities

Files:

- backend/src/entities/User.ts
- backend/src/entities/Session.ts
- backend/src/entities/Payment.ts
- backend/src/entities/Assessment.ts
- backend/src/entities/Therapist.ts
- backend/src/entities/TreatmentPlan.ts
- backend/src/entities/Prescription.ts

Principles:

- Encapsulation.
- Domain-driven design.
- Single responsibility.

Examples:

- Session controls valid status transitions.
- Payment controls refund and paid-state transitions.
- Assessment controls scoring and crisis detection.
- User controls password verification and password update.

User Entity Private Field

File:

- backend/src/entities/User.ts

Principles:

- Encapsulation.
- Information hiding.

passwordHash is private. Other code cannot directly read or mutate it. Code must use methods such as verifyPassword() and setPassword().

Session Lifecycle

File:

- backend/src/entities/Session.ts

Principles:

- State machine.
- Encapsulation.
- Business-rule protection.

The allowed transitions are controlled in one place:

```
PENDING -> CONFIRMED -> ONGOING -> COMPLETED
```

The entity prevents invalid transitions.

Repository Layer

Files:

- backend/src/repositories/UserRepository.ts
- backend/src/repositories/SessionRepository.ts
- backend/src/repositories/PaymentRepository.ts
- backend/src/repositories/TherapistRepository.ts
- backend/src/repositories/AssessmentRepository.ts

Principles:

- Separation of concerns.
- Dependency inversion.
- Persistence abstraction.

Services do not need to know every Prisma query detail.

Service Layer

Files:

- backend/src/services/auth.service.ts
- backend/src/services/session.service.ts
- backend/src/services/payment.service.ts
- backend/src/services/subscription.service.ts
- backend/src/services/complaint.service.ts
- backend/src/services/notification.service.ts

Principles:

- Single responsibility.
- Business logic isolation.
- Transaction management.

Services decide what is allowed and coordinate repositories/providers.

Controllers

Files:

- backend/src/controllers/*.controller.ts

Principles:

- Thin controller pattern.
- Separation of concerns.

Controllers mostly read request values, call services, and return ApiResponse.

API Response Wrapper

File:

- backend/src/Utils/ApiResponse.ts

Principles:

- DRY.
- Consistency.
- Generic typing.

Every successful response has a consistent shape.

API Error Wrapper

File:

- backend/src/Utils/ApiError.ts

Principles:

- Consistency.
- Centralized error handling.
- Explicit error semantics.

Database Service

File:

- backend/src/config/database.ts

Principles:

- Singleton.
- Infrastructure abstraction.

The Prisma client is initialized once and reused.

Redis Token Revocation

File:

- backend/src/services/token.service.ts

Principles:

- Stateless access tokens.
- Stateful refresh-token revocation.
- Security.

Upload Service

File:

- backend/src/services/upload.service.ts

Principles:

- Adapter pattern.
- External provider abstraction.

- Single responsibility.

The app uses Cloudinary without exposing Cloudinary details throughout the codebase.

Stripe Payment Integration

Files:

- backend/src/config/stripe.ts
- backend/src/services/payment.service.ts
- backend/src/controllers/payment.controller.ts

Principles:

- External provider abstraction.
- Webhook-driven architecture.
- Transactional accounting.

Invoice Generation

File:

- backend/src/services/invoice.service.ts

Principles:

- Single responsibility.
- Generated artifact workflow.

Payments can receive generated invoice PDFs.

Notification System

File:

- backend/src/services/notification.service.ts

Principles:

- Adapter pattern.
- Fan-out.
- Best-effort delivery.

The service creates notification records and dispatches email/SMS/push-style channels.

Email Service

File:

- backend/src/services/email.service.ts

Principles:

- Provider abstraction.
- Graceful degradation.

If SMTP is not configured, the service logs and avoids crashing unrelated workflows.

Complaint Workflow

Files:

- backend/src/services/complaint.service.ts
- backend/src/routes/complaint.routes.ts

Principles:

- Workflow state management.
- Role-based authorization.
- Auditability.

Complaints move through statuses such as open, under review, escalated, resolved, and dismissed.

Crisis Monitoring

Files:

- backend/src/entities/Assessment.ts
- backend/src/services/assessment.service.ts
- backend/src/services/adminAnalytics.service.ts

Principles:

- Domain rule encapsulation.
- Event-style notification.
- Admin observability.

Assessments calculate severity and crisis flags. Crisis flags trigger admin alerts and appear in high-risk dashboards.

Real-Time Chat

Files:

- backend/src/socket/chatSocket.ts
- backend/src/services/chat.service.ts
- backend/src/repositories/ChatMessageRepository.ts

Principles:

- Real-time communication.
- Room-based isolation.
- Participant authorization.

Only session participants can join chat rooms.

Frontend API Client

File:

- frontend/src/lib/api.ts

Principles:

- DRY.
- Abstraction.
- Centralized error handling.

All frontend fetch calls share base URL, token, upload, and error behavior.

Frontend Auth Context

File:

- frontend/src/context/AuthContext.tsx

Principles:

- Single source of truth.
- Context provider pattern.
- Session persistence.

Auth state is stored in one provider and exposed through `useAuth()`.

Frontend App Dashboard

File:

- `frontend/src/components/app-dashboard.tsx`

Principles:

- Role-aware UI.
- Component composition.
- Operational interface.

The UI exposes backend workflows depending on whether the user is a client, therapist, or admin.

Frontend Theme System

Files:

- `frontend/src/app/globals.css`
- `frontend/src/components/theme-toggle.tsx`

Principles:

- Design tokens.
- Accessibility.
- Configurable presentation.

Light, dark, and night modes are controlled through CSS variables.

8. Where SOLID Appears In Zenora

Principle | Example In Project | Why It Counts

Single Responsibility | ``AuthService`, `PaymentService`, `UploadService`, `NotificationService`` | Each service owns one domain concern.

Open/Closed | Route modules mounted in ``server.ts`` | New features can be added as new route classes.

Liskov Substitution | Limited inheritance usage | The project avoids unnecessary inheritance, reducing LSP risk.

Interface Segregation | ``Routes`` interface and feature-specific payload interfaces | Code depends on small contracts.

Dependency Inversion | Services receive repository dependencies in constructors | Services are not tightly glued to concrete persistence details.

9. Additional System Design Topics Demonstrated

Authentication And Authorization

Used in:

- `AuthService`
- `AuthMiddleware`
- `AuthRoutes`

Role-Based Access Control

Used across:

- Session routes.

- Therapist routes.
- Admin routes.
- Prescription routes.
- Rating routes.
- Complaint routes.

State Machines

Used in:

- Session
- Payment
- Subscription
- TreatmentPlan
- Complaint status flow.

External Service Integrations

Used for:

- Cloudinary file uploads.
- Stripe payments/subscriptions.
- SMTP email.
- Socket.io chat.

Domain-Driven Modeling

Used in:

- Entity classes.
- Service methods.
- Prisma models.

Cross-Cutting Concerns

Handled by:

- Middleware.
- Logger.
- Error handlers.
- Rate limiter.
- Validation middleware.

Data Consistency

Handled by:

- Prisma transactions.
- Unique constraints.
- Database relations.
- Status transition checks.

API Consistency

Handled by:

- ApiResponse.
- ApiError.

- REST route patterns.
-

10. Important TypeScript Access Modifier Answers

Can a class access its own private property?

Yes.

```
class User {  
  private passwordHash = "abc";  
  
  check() {  
    return this.passwordHash;  
  }  
}
```

Can another class access a private property?

No.

```
const user = new User();  
// user.passwordHash is not allowed
```

Can a subclass access a private property from its parent?

No.

```
class Parent {  
  private secret = "x";  
}  
  
class Child extends Parent {  
  read() {  
    // this.secret is not allowed  
  }  
}
```

Can a subclass access a protected property from its parent?

Yes.

```
class Parent {  
  protected secret = "x";  
}  
  
class Child extends Parent {  
  read() {  
    return this.secret;  
  }  
}
```

Can outside code access a protected property?

No.

```
const child = new Child();  
// child.secret is not allowed
```

Can outside code access a public property?

Yes.

```
class User {  
  public name = "Kunal";  
}  
  
const user = new User();  
console.log(user.name);
```

11. Recommended Improvements For The Project

The project already has a strong layered foundation. Future improvements could include:

- Add integration tests for full request flows.
- Add seed data for demo clients, therapists, and admins.
- Add OpenAPI documentation.
- Add background job queue for notifications and PDF generation.
- Add real SMS and push providers.
- Add refresh-token storage model for stronger session control.
- Add frontend screens specialized per workflow instead of generic operation forms.
- Add observability dashboard with request metrics.
- Add database-level audit logs for admin actions.
- Add stricter domain authorization for therapist access to client records.
- Add end-to-end tests with Playwright.

12. Quick Glossary

Term	Meaning
API	A contract that lets systems communicate.
REST	HTTP-based resource API style.
DTO	Object used to transfer data between layers or over the network.
Entity	Domain object with identity and behavior.
Repository	Object that hides persistence/database access.
Service	Object that contains business logic.
Middleware	Function that runs during request processing.
JWT	Signed token used for authentication.
OAuth	External login protocol used by providers like Google.
ORM	Tool that maps code objects to database tables.
Prisma	Type-safe ORM used in this project.
Webhook	Callback request sent by an external service.

Socket | Persistent real-time connection.

Transaction | Group of database operations that succeed or fail together.

Interface | TypeScript contract for object shape.

Class | Blueprint for objects with state and behavior.

public | Accessible anywhere.

private | Accessible only inside the declaring class.

protected | Accessible inside class and subclasses.

readonly | Cannot be reassigned after initialization.

13. Final Summary

Zenora demonstrates a practical full-stack architecture:

- Next.js frontend with a role-aware operational UI.
- Express backend with clear modules.
- TypeScript domain entities.
- Repository and service layers.
- Prisma relational schema.
- JWT auth and RBAC.
- Cloudinary, Stripe, SMTP, Redis, and Socket.io integrations.
- Clean response/error patterns.
- Validation, transactions, pagination, logging, and admin workflows.

The strongest design choices in this project are separation of concerns, domain entities for important business rules, service-layer orchestration, role-based authorization, consistent validators, and provider abstraction for external systems.

